

SecureEdu: Keyed Audit Chaining and External Checkpoints for Versioned Student Records

Trần Thị Hà Vy

Faculty of Information Technology, Van Hien University, Ho Chi Minh City, Vietnam

VY231A010297@st.vhu.edu.vn

[Co-author/adviser, corresponding author, and ORCID: to be confirmed]

Abstract—This paper presents SecureEdu, a single-node prototype for confidential, versioned student records and authenticated audit logging. The revised design addresses a weakness of an unkeyed local hash chain: a database writer could previously reconstruct a self-consistent history. SecureEdu now derives separate AES-GCM, lookup, and audit domains from a master secret; authenticates every audit block with HMAC-SHA-256; and stores an authenticated chain-head checkpoint outside SQLite. Each create, update, or logical-delete operation atomically commits an encrypted version and its audit event, while an in-process write coordinator preserves consistency between the database head and external checkpoint. Evaluation separates functional mutation tests from adaptive database-writer attacks. The verifier rejected all 390 generated states across 13 attack operators. At 10,000 records, the full profile required 21.14 ms per insertion and 1.525 s for full verification, versus 0.70 ms and 22.60 ms for SQLCipher 4.12. A 1–8 writer experiment showed nearly constant throughput (44.40–48.22 records/s) but increasing tail latency, confirming the single-writer scalability boundary. The result is an authenticated, tamper-evident audit design within an explicit trust boundary—not a distributed blockchain and not protection against compromise of both secret keys and checkpoint storage.

Keywords— *authenticated audit log, AES-GCM, HMAC, student records, tamper evidence, versioned storage.*

I. INTRODUCTION

Student information systems store identity data, enrolment information, grades, and academic history. Conventional access control restricts legitimate application users, while encryption at rest limits disclosure if storage media are copied. Neither control alone provides a trustworthy explanation of how a record changed over time. A privileged database writer may alter current rows or audit metadata outside the application path.

Earlier SecureEdu work combined AES-GCM, keyed lookup tokens, retained versions, and a SHA-256-linked audit chain. Review exposed a decisive flaw: because the chain used only public hashing and its head remained inside the same database, an adaptive writer could delete or reorder valid encrypted versions, recompute every subsequent hash, repair the local head, and present a self-consistent database. Simple byte-flip experiments therefore demonstrated consistency checking, not tamper evidence against that attacker.

This revision asks a narrower research question: what is the smallest single-node design that detects database-only history reconstruction while preserving record confidentiality and operational simplicity? It makes three contributions. First, it defines an authenticated audit construction in which record encryption, exact-match lookup, block authentication, and checkpoint authentication use domain-separated key material. Second, it implements and tests an external authenticated checkpoint plus a coordinated write boundary spanning SQLite commit and checkpoint refresh. Third, it evaluates adaptive attacks and writer contention separately from ordinary corruption and reports the limits of each experiment. No new cryptographic primitive is claimed; the contribution is a threat-driven composition and

falsifiable evaluation of standard mechanisms for a constrained educational-record workload.

II. RELATED WORK AND DESIGN GAP

SQLCipher transparently encrypts SQLite pages and authenticates them per page, which is well suited to confidentiality of a database file [1]. It does not by itself define application-level record versions, actor-bound events, or an externally witnessed history. SecureEdu instead encrypts individual payloads and retains application semantics, at the cost of more schema and transaction logic.

Schneier and Kelsey study protected logs on an untrusted machine and use evolving secrets plus remote interaction to limit undetectable modification after compromise [2]. Crosby and Wallach develop Merkle-tree history structures with logarithmic proofs and explicit auditing semantics [3]. Certificate Transparency likewise combines append-only Merkle trees, consistency proofs, and signed tree heads [4]. These systems provide stronger proof and witness models than SecureEdu, but also solve broader distributed-log problems. SecureEdu uses a linear authenticated chain with $O(n)$ full verification because its target is a small, single-institution prototype.

Educational blockchain research commonly emphasizes portable credentials, learner ownership, or multi-party verification [5], [6]. Those goals require distributed governance that is absent here. SecureEdu protects operational record history inside one administrative application. Table I clarifies the comparison: the SQLite and AES profiles in Section V are ablations for measuring incremental cost, not independent security baselines.

Table I. Qualitative comparison of protection models

System	Confidential storage	Authenticated history	External freshness evidence	Verification
SQLCipher [1]	page level	no application history	no	page access
Schneier–Kelsey [2]	log entries	forward-secure MAC construction	remote verifier	sequential
Crosby–Wallach / CT [3], [4]	not primary goal	Merkle append-only log	signed/witnessed roots	logarithmic proofs
SecureEdu	AES-GCM per version	HMAC linear chain	local authenticated checkpoint	O(n) full scan

III. THREAT MODEL AND SECURITY DESIGN

A. Scope and adversaries

The prototype runs on one host and stores data in SQLite. Roles are administrator, registrar, and auditor. The database-only adversary may read and write SQLite files, ciphertext envelopes, version pointers, audit rows, indexes, and ordering. This adversary does not possess the master secret and cannot modify the application code or the external checkpoint file. A stronger host adversary that obtains both the secret and checkpoint can construct an accepted history and is out of scope. Denial of service, malicious but authorized input, endpoint compromise, and network confidentiality are also outside the claimed boundary.

The checkpoint must therefore be protected independently in deployment—for example by operating-system permissions, a KMS-backed signature, WORM storage, or a remote witness. In the prototype it is a separate authenticated file on the same machine. This detects the modeled database-only writer but is not independent physical storage; the distinction is material.

B. Authenticated record envelope

Each normalized student record is serialized as canonical UTF-8 JSON and encrypted with AES-256-GCM [7]. A fresh 96-bit nonce is generated for every version. Associated data binds the schema version, internal record identifier, version number, operation, actor identifier, and actor role. Moving a valid ciphertext to another context therefore invalidates its authentication tag. SQLite also enforces uniqueness of $(key_id, nonce)$.

The normalized student code is represented by

$$T = \text{HMAC-SHA-256}(K_{\text{lookup}}, \text{normalized_code}).$$

This supports exact-match lookup without storing the identifier in plaintext. Equality leakage remains, and compromise of K_{lookup} enables enumeration of low-entropy codes. HKDF domain separation derives independent lookup and audit keys from the configured master secret [8].

C. Keyed audit chain and external checkpoint

For version event i , SecureEdu hashes a canonical block containing the index, timestamp, record and version identifiers, operation, actor data, envelope hash, and previous block hash. It then computes

$$M_i = \text{HMAC-SHA-256}(K_{\text{audit}}, \text{domain_block} || i || H_i).$$

The next block covers H_i , and the verifier recomputes both the public digest and keyed MAC. A database writer without K_{audit} can rehash public fields but cannot produce valid block MACs. The authenticated checkpoint stores (i, H_i, M_i) plus

$$C_i = \text{HMAC-SHA-256}(K_{\text{audit}}, \text{domain_checkpoint} || i || H_i || M_i).$$

The checkpoint is compared with the latest database block before every write and during full verification. It detects rollback, suffix truncation, and a reconstructed local head within the stated separation assumption. Domain strings prevent cross-protocol substitution between lookup tokens, blocks, and checkpoints [9].

D. Atomicity and concurrency boundary

One `BEGIN IMMEDIATE` transaction writes the immutable encrypted version, appends its audit block, and updates the current-record pointer. SQLite permits one simultaneous writer [10]. The checkpoint necessarily resides outside that transaction, creating a crash window between database commit and checkpoint replacement. SecureEdu writes checkpoints through a temporary file, flushes it, and atomically replaces the old file. A process-wide re-entrant lock now spans database verification, commit, and checkpoint refresh for all service instances targeting the same path. This prevents threads in one Python process from observing an intermediate head. It does not coordinate multiple operating-system processes and is not a distributed commit protocol.

IV. IMPLEMENTATION AND VERIFICATION

The Python/Flask implementation separates web authorization from `RecordService`, the sole coordinator for cryptographic and persistence operations. The SQLite schema contains record heads, immutable encrypted versions, authenticated audit blocks, and users. Logical deletion appends a `DELETE` version instead of erasing history. Optimistic version checks reject stale updates. Passwords use script; state-changing forms use CSRF tokens; repeated login failures trigger temporary lockout; and routes enforce role permissions.

Full verification checks: (1) the external checkpoint MAC and equality with the latest block; (2) genesis, indexes, previous links, block hashes, and block HMACs; (3) a one-to-one mapping between versions and audit blocks; (4) record-version continuity and legal operation sequences; (5) envelope digests and AES-GCM tags; and (6) lookup tokens for current active records. A failure report identifies the violated layer but does not attempt automatic repair.

The current database schema is version 4, while authenticated audit blocks use block schema 3. Keeping these version namespaces distinct avoids the earlier ambiguous phrase “schema v3.” The test suite contains 118 tests and reaches 90.90% statement coverage. Coverage supports implementation confidence but is neither a security proof nor an estimate of attack-detection probability.

V. EVALUATION METHOD

Experiments ran on Windows 11 with an Intel Core i5-10300H CPU, 7.84 GiB RAM, Python 3.12.14, SQLite 3.53.1, Flask 3.1.3, and cryptography 49.0.0. Synthetic records were deterministically generated with seed 2026 and contain no real student data. Scripts preserve raw CSV, descriptive statistics, commit identifiers, and a SHA-256 digest of executable source files.

Four configurations are measured. P1 stores plaintext JSON in SQLite; B1 uses the independent SQLCipher 4.12 Community baseline with a raw 256-bit key, default page-authentication settings, WAL, and `cipher_integrity_check`; P2 stores application-level AES-GCM envelopes; and P3 uses the complete SecureEdu version, lookup, authenticated-chain, and checkpoint path. P1 and P2 are controlled ablations, whereas B1 provides a deployable encrypted-SQLite baseline. B1 protects pages but does not implement P3’s version-history semantics. At 100 and 1,000 records, P1–P3 were repeated 10 times with shuffled order; the 10,000-record P1–P3 run was repeated three times because each P3 run requires several minutes. B1 was repeated 10 times at every size. Every record uses its own transaction to match the interactive service path. Timing uses

Table II. Mean cost by profile (10 repetitions except P1–P3 at n = 10,000: 3)

Profile	n	Insert (ms/record)	Full verify (ms)	DB size (MiB)
P1 SQLite	100	0.56	0.03	0.047
B1 SQLCipher 4.12	100	0.69	0.07	0.051
P2 SQLite + AES-GCM	100	0.62	0.82	0.059
P3 SecureEdu	100	20.67	24.71	0.211
P1 SQLite	1,000	0.65	0.49	0.383
B1 SQLCipher 4.12	1,000	0.78	2.28	0.391
P2 SQLite + AES-GCM	1,000	0.78	8.38	0.482
P3 SecureEdu	1,000	26.76	213.03	1.464
P1 SQLite	10,000	0.56	1.62	3.695
B1 SQLCipher 4.12	10,000	0.70	22.60	3.766
P2 SQLite + AES-GCM	10,000	0.73	75.06	4.652
P3 SecureEdu	10,000	21.14	1,525.28	14.155

At 10,000 records, P3 insertion is 30.2 times B1, its database is 3.76 times larger, and its full verification is 67.5 times slower. These ratios quantify the cost of application versions and authenticated history relative to transparent encrypted SQLite; they are not like-for-like security rankings. P3 includes canonical serialization, lookup HMAC, immutable version storage, audit authentication, two SQLite connections, and durable checkpoint replacement. Verification workloads also differ: B1 checks page authentication, P2 authenticates every record ciphertext, and P3 additionally validates the complete relational and audit structure. The independent baseline resolves the absence of comparison in the earlier evaluation while preserving these semantic qualifications.

Table III. In-process writer contention (120 records, 10 repetitions)

Threads	Throughput (records/s)	Median latency (ms)	p95 latency (ms)
1	46.48	20.93	27.70
2	47.15	40.49	59.59
4	44.40	83.78	147.90
8	48.22	154.76	230.68

Throughput remains approximately flat while median and p95 latency grow with the number of writers. This is the expected consequence of serialized SQLite writes plus

`perf_counter_ns` on persistent Windows storage, not `tmpfs`.

Security tests create a valid three-event history, mutate a copied database below the service layer, and invoke the independent verifier. Six operators introduce inconsistent corruption: ciphertext, authentication tag, nonce, envelope digest, previous link, and middle-block deletion. Seven adaptive database-writer operators repair public dependent state: modify-and-rehash, timestamp-and-rehash, delete-and-rechain, truncate-and-repair-head, splice valid history, reorder/reindex, and partial per-record rollback. Each operator is repeated 30 times. Because the expected result follows from key possession assumptions, counts validate implementation coverage, not cryptographic detection probability.

The contention experiment fixes 120 inserted records, varies 1, 2, 4, and 8 threads, and repeats each level 10 times. Each thread uses its own `RecordService` instance against one database; the shared in-process lock and final full verification are active.

VI. RESULTS AND DISCUSSION

A. Layered performance

B. Adaptive-attack tests

All 390 generated states were rejected: 180/180 inconsistent mutations and 210/210 adaptive database-writer cases. The revised result closes the concrete modify-and-rehash counterexample only under the assumption that the attacker lacks `K_audit` and cannot replace the authenticated checkpoint. If the same attacker obtains the secret and checkpoint, block and checkpoint MACs can be regenerated. The experiment therefore validates that the implemented verifier enforces its construction; it does not establish unconditional immutability.

C. Writer contention

the checkpoint critical section. The result supports small administrative workloads, but not horizontal write scaling. Multiple WSGI processes require an inter-process lock or

a transactional external witness; they are not supported by the current coordinator.

VII. LIMITATIONS AND THREATS TO VALIDITY

The checkpoint and database are separate resources. Atomic file replacement reduces torn writes but cannot remove the post-commit/pre-checkpoint crash window. A recovery protocol must distinguish a crash from tampering without silently blessing a rolled-back database. The audit key is derived from the same configured master secret, so host-level key disclosure defeats all keyed audit claims and provides no forward security. Production deployment should separate keys using a KMS/HSM and place signed checkpoints in independent WORM or transparency storage.

External validity is limited by synthetic records, one Windows laptop, one SQLite build, small datasets, and one process. Cache state and background tasks contribute to variance. Per-record transactions reflect interactive writes but penalize bulk imports. P1/P2 are controlled ablations, while SQLCipher is an independent page-encryption baseline. SQLCipher and SecureEdu still differ materially in history semantics, so their ratios measure implementation cost rather than equivalent security. Merkle logs and managed databases remain qualitative comparisons because their proof, deployment, and trust models are not directly interchangeable.

The mutation suite covers named attack transformations on a short generated history. It is not formal verification, penetration testing, or evidence for a probabilistic “100% secure” claim. The web controls have automated coverage but the prototype lacks production HTTPS termination, managed secrets, key rotation, backup/restore policy, privacy retention analysis, and multi-process coordination.

VIII. CONCLUSION

SecureEdu replaces an inadequate public hash chain with domain-separated block HMACs and an authenticated external checkpoint, while retaining AES-GCM encrypted record versions and atomic SQLite event writes. The revision is important because it changes the security argument: database consistency alone is insufficient; tamper evidence requires a secret or witness outside the attacker-controlled state. The implementation rejects the tested adaptive reconstructions, but its guarantee stops when both audit key and checkpoint are compromised. Measured contention also confirms that the current single-node design scales by bounded workload, not by concurrent writers. Future work should add a KMS-held signing key, remote witnessed checkpoints, crash-safe recovery, forward-secure key evolution, and independent encrypted-database baselines before any multi-node or “blockchain” claim.

ACKNOWLEDGMENT

[Adviser/co-author decision, acknowledgment, and funding information: to be confirmed.]

REFERENCES

- [1] Zetetic LLC, “SQLCipher design: Security approach and features,” 2026. [Online]. Available: <https://www.zetetic.net/sqlcipher/design/>. Accessed: Sep. 27, 2026.
- [2] B. Schneier and J. Kelsey, “Cryptographic support for secure logs on untrusted machines,” in *Proc. 7th USENIX Security Symp.*, San Antonio, TX, USA, 1998.
- [3] S. A. Crosby and D. S. Wallach, “Efficient data structures for tamper-evident logging,” in *Proc. 18th USENIX Security Symp.*, Montreal, QC, Canada, 2009, pp. 317–334.
- [4] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, Dec. 2021, doi: 10.17487/RFC9162.
- [5] M. Kataev and L. Bulysheva, “Blockchain system in higher education: Storing academic students’ records and achievements accumulated in the educational process,” *Syst. Res. Behav. Sci.*, vol. 39, no. 3, pp. 589–596, 2022, doi: 10.1002/sres.2872.
- [6] N. Smolenski, “Blockchain for education: A new credentialing ecosystem,” in *OECD Digital Education Outlook 2021*. Paris, France: OECD Publishing, 2021, doi: 10.1787/589b283f-en.
- [7] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.
- [8] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, May 2010, doi: 10.17487/RFC5869.
- [9] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.
- [10] SQLite Consortium, “Transaction,” *SQLite Documentation*, 2026. [Online]. Available: https://www.sqlite.org/lang_transaction.html. Accessed: Sep. 27, 2026.
- [11] T. T. H. Vy, “SecureEdu authenticated student records: source code and reproducibility artifacts,” GitHub repository, 2026. [Online]. Available: <https://github.com/hv6z/secureedu-authenticated-student-records>. Accessed: Sep. 27, 2026.